

Helix OTA — Memory-leak discriminator fix (whole-branch-review remediation)

Revision: 1 **Last modified:** 2026-07-10T00:05:00Z **Scope:** server/internal/api/memory_test.go. **Authority:** §11.4.4 (STOP-and-fix on discovery), §11.4.6 (no-guessing / no overclaim), §11.4.107(10) (self-validated analyzer), §11.4.115 (RED reproduces the real defect signal), §11.4.120 (reconcile a fix vs its own gate, never fake-pass), §11.4.123 (rock-solid proof — use the robust method, don't downgrade the claim), §11.4.142/§11.4.165 (independent whole-branch review).

1. The finding (independent whole-branch review, docs/research/session_wholebranch_review_20260710/REVIEW.md, Important-1)

The TestMemory_SustainedAPILoadNoGrowth guard added in commit f82a77e4 computed its threshold from per-equal-batch **deltas**:

```
ref          = max(growth1, growth2)    // growth of two equal-size
batches
threshold = ref * 4                     // floored at 512 KiB
FAIL iff growth3 > threshold
```

This is **structurally blind to a steady linear leak**: a handler leaking L bytes/request retains $\sim \text{batchSize} \cdot L$ in every equal-size batch, so $\text{growth1} \approx \text{growth2} \approx \text{growth3} \approx \text{batchSize} \cdot L$. Then $\text{ref} = \text{batchSize} \cdot L$, $\text{threshold} = 4 \cdot \text{batchSize} \cdot L$, and $\text{growth3} = \text{batchSize} \cdot L < \text{threshold}$ — **always passes**. Only the 512 KiB absolute floor caught anything, so a leak below ~ 350 B/req/batch passed undetected, while the commit/

evidence claimed it detects “a genuine per-request heap leak.” Per §11.4.6 that is an **overclaim**; the §11.4.115 polarity proof it carried only showed the assertion *wiring* fires, not that a leak is *detected*.

2. The fix — retention-SCALES-WITH-LOAD discriminator (§11.4.123 robust method)

A genuine steady leak has one load-invariant signature: **cumulative retained live heap scales with the number of requests served**; a healthy handler’s post-GC live heap **plateaus** after warmup. The new test measures retained heap (post-runtime.GC() HeapAlloc, relative to a warmed baseline) at two very different cumulative request counts — a small phase ($N = 1500$) and a large phase ($\sim 8 \times N$) — and delegates the verdict to a pure, unit-testable function:

```
func classifyHeapGrowth(retainedSmall, retainedLarge, noiseFloor
    int64, scaleNum, scaleDen int64) (leak bool, reason
    string)
// leak  $\Leftrightarrow$  retainedLarge > noiseFloor AND retainedLarge >
// ref*(scaleNum/scaleDen)
// where ref = max(retainedSmall, noiseFloor) (never
// noise+noise)
```

A steady leak $\rightarrow$ retainedLarge $\approx 8 \times$ retainedSmall $\rightarrow$ exceeds ref*3 $\rightarrow$ **leak**. A healthy handler $\rightarrow$ retainedLarge $\approx$ retainedSmall (or below floor) $\rightarrow$ **no leak**. This catches the sub-floor-per-batch steady leak the delta test missed, because the signal is the **scaling across 8× the requests**, not any single batch size.

3. Proof (real captured evidence — go test

```
-run 'TestMemoryGrowthClassifier|
TestMemory_SustainedAPILoadNoGrowth'
-count=1 -v ./internal/api/)
```

Test	What it proves	Result
TestMemory_SustainedAPILoadNoGrowth (real Gin router, real store, no mocks)	healthy handler is NOT false-flagged	PASS — retainedSmall=13B, retainedLarge=-2

Test	What it proves	Result
		B → flat/below floor → leak=false
TestMemoryGrowthClassifier (7 golden cases + explicit §11.4.107(10) gate)	analyzer passes golden-good AND flags golden-bad; boundary at exactly ref*3 correct	PASS
TestMemoryGrowthClassifier_DetectsInjectedSteadyLeak (§11.4.115 RED — real 4096 B/iter leak retained in a growing sink)	detection genuinely works end-to-end	PASS — retainedSmall≈8. MB, retainedLarge≈65 MB (~8×) → leak=true DETECTED

Package result: ok github.com/HelixDevelopment/helix_ota/server/internal/api 0.190s. gofmt clean, go vet ./internal/api/ clean.

The injected-leak RED is the key difference from the old design: it retains a real, growing amount of live heap (65 MB across the large phase) and the SAME classifier the real test uses flags it — proving **leak DETECTION**, not merely assertion wiring. An analyzer that failed to flag this injected leak would be a §11.4 bluff; it does not.

4. Honest boundary (§11.4.6)

- The healthy-handler numbers (1.3 KB / -0.2 KB retained over ~12,000 GETs) show the real core-API list handlers plateau — no leak, as expected for read-only reads over a fixed dataset. This is a real PASS, not an absence-of-error PASS.
- The discriminator detects leaks whose retention SCALES with load. A one-time fixed allocation (grows once, then plateaus) is correctly NOT flagged (golden case one-time-growth-no-scaling) — that is not a leak. A leak that saturates a bounded cache is out of scope by

construction (it plateaus); this guard targets unbounded per-request retention, which is the dangerous class.

- Corrects the f82a77e4 overclaim going forward (that commit message is immutable under §11.4.113 no-force-push); this file + the new code comments state precisely what is and is not detected.

Sources verified

Sources verified 2026-07-10: - Go `runtime.MemStats` (HeapAlloc, GC semantics) — <https://pkg.go.dev/runtime#MemStats> - Go testing benchmarks/leak patterns — <https://pkg.go.dev/testing>